UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e INTERACCIÓN HUMANO COMPUTADORA

REPORTE DE PRÁCTICA Nº 4

NOMBRE COMPLETO: Pérez Uribe José Alberto

Nº de Cuenta: 318143213

GRUPO DE LABORATORIO: 02

GRUPO DE TEORÍA: 07

SEMESTRE 2026-2

FECHA DE ENTREGA LÍMITE: 17/033/2026

CALIFICACIÓN: _____

Ejercicios de practica

1.- Terminar la Grúa con:

-cuerpo (prisma rectangular)

-base (pirámide cuadrangular)

-4 llantas (4 cilindros) con teclado se pueden girar cada una de las 4 llantas por separado

Para realizar esta continuación del ejercicio de clase primero se declaran las variables que permiten controlar el tiempo, la cámara, la ventana, los objetos geométricos y los shaders.

```
const float PI = 3.14159265f;
GLfloat deltaTime = 0.0f;
GLfloat lastTime = 0.0f;
static double limitFPS = 1.0 / 60.0;

Camera camera;
Window mainWindow;
vector<Mesh*> meshList;
vector<Shader> shaderList;

static const char* vShader = "shaders/shader.vert";
static const char* fShader = "shaders/shader.frag";

Sphere sp = Sphere(1.0, 20, 20);
```

En esta otra parte se define un arreglo donde se guarda el ángulo de giro de cada llanta por separado. Esto es fundamental porque permite que las cuatro ruedas no dependan de una sola rotación, sino que cada una pueda moverse de manera independiente con distintas teclas.

```
float giroLlantas[4] = { 0.0f, 0.0f, 0.0f, 0.0f };
```

Gracias a esto, más adelante, cuando se renderiza cada rueda, se le aplica su propio ángulo de giro.

Aquí se crea la función encargada de modificar los valores del arreglo giroLlantas según las teclas presionadas. Esta parte es importante porque conecta la interacción del teclado con el movimiento visual de las ruedas.

```
void ControlLlantas(bool* keys, float dt)
{
    float vel = 120.0f * dt;

    // Llanta 1
    if (keys[GLFW_KEY_1]) giroLlantas[0] += vel;
    if (keys[GLFW_KEY_Z]) giroLlantas[0] -= vel;

    // Llanta 2
    if (keys[GLFW_KEY_2]) giroLlantas[1] += vel;
    if (keys[GLFW_KEY_X]) giroLlantas[1] -= vel;

    // Llanta 3
    if (keys[GLFW_KEY_3]) giroLlantas[2] += vel;
    if (keys[GLFW_KEY_C]) giroLlantas[2] -= vel;

    // Llanta 4
    if (keys[GLFW_KEY_4]) giroLlantas[3] += vel;
    if (keys[GLFW_KEY_V]) giroLlantas[3] -= vel;
}
```

- se calcula una velocidad angular con dt, para que el giro sea uniforme aunque cambien los FPS;
- cada rueda tiene dos teclas: una para girar en un sentido y otra para el contrario;

En esta función se define la geometría del cubo mediante vértices e índices. Esta figura es una de las más importantes del modelo, ya que después se reutiliza para formar varias partes: el cuerpo principal, los brazos mecánicos y la canasta.

```
void CrearCubo()
{
    unsigned int cubo_indices[] = {
        0, 1, 2, 2, 3, 0,
        1, 5, 6, 6, 2, 1,
        7, 6, 5, 5, 4, 7,
        4, 0, 3, 3, 7, 4,
        4, 5, 1, 1, 0, 4,
        3, 2, 6, 6, 7, 3
    };

    GLfloat cubo_vertices[] = {
        -0.5f, -0.5f,  0.5f,
         0.5f, -0.5f,  0.5f,
         0.5f,  0.5f,  0.5f,
        -0.5f,  0.5f,  0.5f,
        -0.5f, -0.5f, -0.5f,
         0.5f, -0.5f, -0.5f,
         0.5f,  0.5f, -0.5f,
        -0.5f,  0.5f, -0.5f
    };

    Mesh* cubo = new Mesh();
    cubo->CreateMesh(cubo_vertices, cubo_indices, 24, 36);
    meshList.push_back(cubo);
}
```

Esta función genera la geometría del cilindro de forma procedural. Es importante porque el cilindro se usa posteriormente para representar las llantas.

```
void CrearCilindro(int res, float R)
{
    int n, i;
    GLfloat dt = 2 * PI / res, x, z, y = -0.5f;

    vector<GLfloat> vertices;
    vector<unsigned int> indices;

    for (n = 0; n <= res; n++) {
        if (n != res) {
            x = R * cos((n)*dt);
            z = R * sin((n)*dt);
        }
        else {
            x = R * cos(0);
            z = R * sin(0);
        }

        for (i = 0; i < 6; i++) {
            switch (i) {
            case 0: vertices.push_back(x); break;
            case 1: vertices.push_back(y); break;
            case 2: vertices.push_back(z); break;
            case 3: vertices.push_back(x); break;
            case 4: vertices.push_back(0.5f); break;
            case 5: vertices.push_back(z); break;
            }
        }
    }
```

```
    for (n = 0; n <= res; n++) {
        x = R * cos((n)*dt);
        z = R * sin((n)*dt);
        vertices.push_back(x);
        vertices.push_back(-0.5f);
        vertices.push_back(z);
    }

    for (n = 0; n <= res; n++) {
        x = R * cos((n)*dt);
        z = R * sin((n)*dt);
        vertices.push_back(x);
        vertices.push_back(0.5f);
        vertices.push_back(z);
    }

    for (i = 0; i < (int)vertices.size(); i++) indices.push_back(i);

    Mesh* cilindro = new Mesh();
    cilindro->CreateMeshGeometry(vertices, indices, vertices.size(), indices.size());
    meshList.push_back(cilindro);
}
```

Aquí se define otra geometría base: una pirámide de base cuadrada. Esta pieza se utiliza después como la base del vehículo.

```
void CrearPiramideCuadrangular()
{
    vector<unsigned int> indices = {
        0,3,4,
        3,2,4,
        2,1,4,
        1,0,4,
        0,1,2,
        0,2,3
    };

    vector<GLfloat> vertices = {
         0.5f,-0.5f, 0.5f,
         0.5f,-0.5f,-0.5f,
        -0.5f,-0.5f,-0.5f,
        -0.5f,-0.5f, 0.5f,
         0.0f, 0.5f, 0.0f,
    };

    Mesh* piramide = new Mesh();
    piramide->CreateMeshGeometry(vertices, indices, 15, 18);
    meshList.push_back(piramide);
}
```

Esta figura es importante porque le da una base al modelo y evita que todo esté construido solo con prismas rectangulares.

Aquí se crea la transformación base de todo el objeto. Es decir, se define la posición general del vehículo dentro de la escena.

```
model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(0.0f, -1.0f, -10.0f));
modelaux = model;
```

Esta parte funciona como el punto de partida de toda la jerarquía. A partir de este nodo se construyen las demás piezas.

Aquí se coloca y escala la pirámide que sirve como base del modelo.

```
model = modelaux;
model = glm::translate(model, glm::vec3(0.0f, 2.0f, 0.0f));
nodoPiramide = model;

model = glm::scale(model, glm::vec3(4.5f, 3.0f, 3.0f));
color = glm::vec3(0.78f, 0.78f, 0.78f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshList[2]->RenderMeshGeometry();
```

Esto es relevante porque la pirámide se convierte en el primer gran bloque visible del modelo, y además nodoPiramide se guarda para usarlo como referencia en otras piezas, como las llantas y el cuerpo.

En esta sección se definen las posiciones de las cuatro ruedas y se usa un ciclo para dibujarlas una por una.

```
float ruedaX[4] = { -2.2f,  2.2f, -2.2f,  2.2f };
float ruedaZ[4] = { 1.5f,  1.5f, -1.5f, -1.5f };

for (int i = 0; i < 4; i++) {
    model = nodoPiramide;
    model = glm::translate(model, glm::vec3(ruedaX[i], -2.0f, ruedaZ[i]));

    model = glm::rotate(model, glm::radians(90.0f), glm::vec3(0.0f, 0.0f, 1.0f));
    model = glm::rotate(model, glm::radians(giroLlantas[i]), glm::vec3(1.0f, 0.0f, 0.0f));

    model = glm::scale(model, glm::vec3(0.85f, 0.40f, 0.85f));
    color = glm::vec3(0.08f, 0.08f, 0.08f);
    glUniform3fv(uniformColor, 1, glm::value_ptr(color));
    glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
    meshList[1]->RenderMeshGeometry();
}
```

Después se construye el cuerpo del vehículo a partir de un cubo escalado.

```
model = nodoPiramide;
model = glm::translate(model, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion1()), glm::vec3(0.0f, 1.0f, 0.0f));
modelaux = model;

model = glm::scale(model, glm::vec3(4.0f, 2.5f, 2.5f));
color = glm::vec3(1.0f, 0.82f, 0.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshList[0]->RenderMesh();
```

A partir de la cabina construí un sistema de brazos articulados utilizando transformaciones jerárquicas. Cada articulación se posiciona trasladando el sistema al extremo del brazo anterior y posteriormente aplicando una rotación sobre el eje z controlada por una variable de articulación. Esta transformación se guarda en una matriz auxiliar para que el siguiente segmento herede correctamente la posición y orientación del anterior.

```
model = modelaux;
model = glm::translate(model, glm::vec3(2.0f, 0.7f, 0.0f));
model = glm::rotate(model, glm::radians(35.0f + mainWindow.getarticulacion2()), glm::vec3(0.0f, 0.0f, 1.0f));
modelaux = model;
```

Para representar visualmente cada punto de giro utilicé una esfera escalada, lo que permite identificar claramente la unión entre los distintos segmentos del brazo.

```
model = glm::scale(model, glm::vec3(0.35f, 0.35f, 0.35f));
sp.render();
```

A partir de cada articulación se genera un nuevo brazo utilizando nuevamente el cubo como geometría base, aplicando una traslación para posicionarlo y una escala para darle la longitud correspondiente.

```
model = modelaux;
model = glm::translate(model, glm::vec3(2.5f, 0.0f, 0.0f));
modelaux2 = model;
model = glm::scale(model, glm::vec3(5.0f, 0.6f, 0.6f));
meshList[0]->RenderMesh();
```

Este mismo procedimiento se repite para las articulaciones siguientes, utilizando matrices auxiliares para conservar el punto exacto donde termina cada brazo y desde donde comienza el siguiente. Gracias a esta estructura jerárquica, cada segmento hereda las transformaciones del anterior, lo que permite que todo el sistema del brazo se mueva de forma coherente cuando se modifican los ángulos de las articulaciones. La canasta la construí reutilizando el cubo varias veces.

En lugar de crear una geometría nueva, usé distintas instancias escaladas y trasladadas para representar el piso, los postes verticales y las barras superiores. Todas estas piezas parten de la transformación almacenada en modelaux, lo que garantiza que la canasta completa herede correctamente los movimientos de la articulación final.

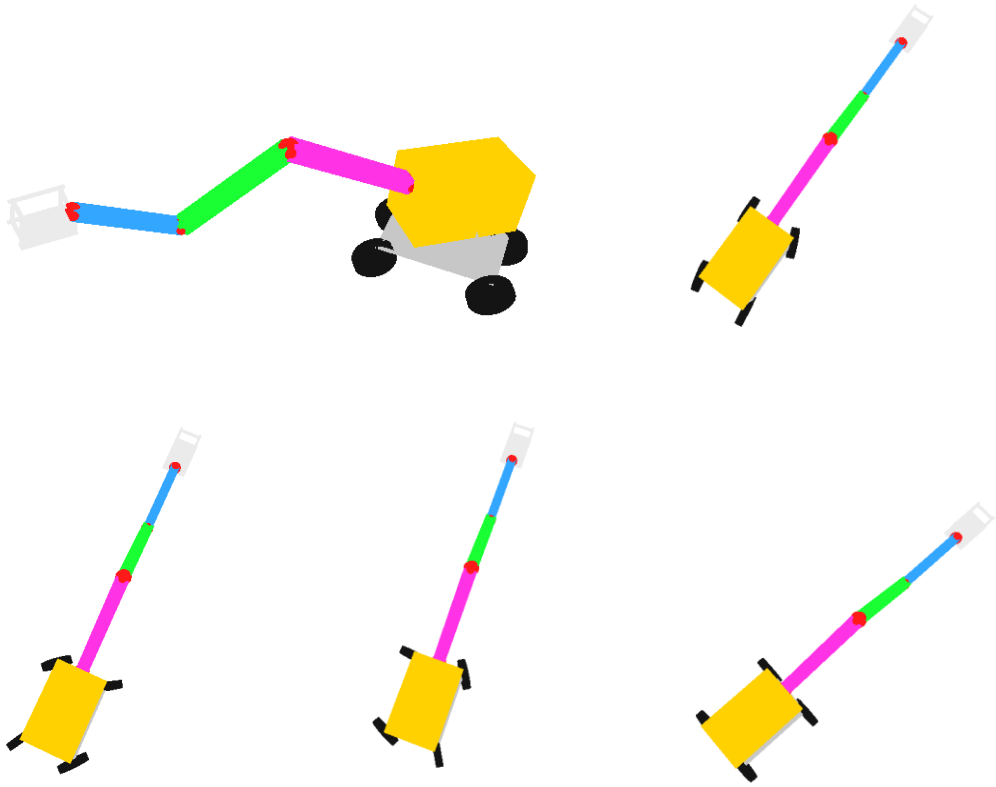
```
model = modelaux;
model = glm::translate(model, glm::vec3(0.9f, -0.35f, 0.0f));
model = glm::scale(model, glm::vec3(1.8f, 0.2f, 1.2f));
color = glm::vec3(0.92f, 0.92f, 0.92f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshList[0]->RenderMesh();
```

Con esta organización logré construir una grúa jerárquica compuesta por una base piramidal, cuatro llantas, una cabina giratoria, varios brazos articulados y una canasta al final. Lo que hace que todo funcione correctamente es el uso ordenado de matrices auxiliares para conservar nodos intermedios, la reutilización de primitivas geométricas y la aplicación encadenada de traslaciones, rotaciones y escalas. Gracias a eso, cada parte queda unida a la anterior y responde de forma coherente cuando se modifica alguna articulación.

El giro de cada llanta se controla con el teclado de forma independiente:

- Llanta 1: 1 (gira) / Z (contrario)
- Llanta 2: 2 / X
- Llanta 3: 3 / C
- Llanta 4: 4 / V

Esto permite manipular cada rueda por separado y en ambos sentidos.



Como resultado final, se obtiene un modelo 3D articulado funcional que representa una grúa con múltiples grados de libertad. A través del uso de primitivas básicas y transformaciones jerárquicas, se logra que cada parte del sistema (base, llantas y brazo mecánico) se comporte de manera independiente pero coordinada. Además, la interacción mediante teclado permite manipular las articulaciones y el movimiento de las ruedas en tiempo real, demostrando correctamente los conceptos de modelado geométrico y jerarquías en OpenGL.

2.- Crear un animal robot 3d

-Instanciando cubos, pirámides, cilindros, conos, esferas:

-4 patas articuladas en 2 partes (con teclado se puede mover las dos articulaciones de cada pata)

-cola articulada o 2 orejas articuladas. (con teclado se puede mover la cola o cada oreja independiente1.- Ejecución de los ejercicios que se dejaron, comentar cada uno y capturas de pantalla de bloques de código generados y de ejecución del programa.

Primero volvemos a usar las primitivas que ya hemos visto a lo largo de la practica:

```
CrearCubo();
CrearPiramideTriangular();
CrearCilindro(20, 0.5f);
CrearCono(20, 0.5f);
CrearPiramideCuadrangular();
CreateShaders();
```

Estas funciones permiten simplificar el modelado, ya que en lugar de definir cada figura desde cero, se emplean como bloques base que después se transforman para formar estructuras más complejas.

Después se definen las variables que controlan las articulaciones del modelo, específicamente las patas y la cola, permitiendo almacenar los ángulos de rotación de cada segmento.

```
float pataSuperior[4] = { 10.0f, -10.0f, -8.0f, 8.0f };
float pataInferior[4] = { -20.0f, -20.0f, -20.0f, -20.0f };
float colaAng = 20.0f;
float colaAng2 = 10.0f;
```

Estas variables son importantes porque representan el estado actual del movimiento del perro, es decir, determinan cómo están posicionadas sus patas y su cola en cada momento.

A continuación se implementa la función que actualiza dichas articulaciones con base en la entrada del usuario, permitiendo modificar los ángulos mediante el teclado.

```
void ActualizarArticulaciones(bool* keys, float dt)
{
    float vel = 80.0f * dt;

    if (keys[GLFW_KEY_KP_7]) pataSuperior[0] += vel;
    if (keys[GLFW_KEY_KP_8]) pataSuperior[0] -= vel;

    if (keys[GLFW_KEY_F]) colaAng += vel;
    if (keys[GLFW_KEY_V]) colaAng -= vel;
}
```

Aquí se conecta directamente la interacción del teclado con el modelo, ya que al presionar teclas se modifican los valores de rotación, lo que posteriormente se verá reflejado en el movimiento de las extremidades.

En la parte principal del programa se inicializa la ventana, la cámara y las figuras, estableciendo el entorno donde se va a dibujar el modelo. Dentro del ciclo principal se actualiza constantemente la escena, calculando el tiempo entre frames, leyendo la entrada del usuario y limpiando la pantalla para volver a dibujar.

Posteriormente se define un nodo base que funciona como referencia para construir todo el cuerpo del perro, asegurando que todas las partes estén correctamente posicionadas en el espacio.

```
glm::mat4 model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(0.0f, -1.0f, -8.0f));
glm::mat4 nodoCuerpo = model;
```

Este nodo es fundamental porque a partir de él se derivan todas las demás transformaciones del modelo.

A partir de este nodo se construye el cuerpo y la cabeza, aplicando escalados y traslaciones para dar forma al perro.

```
glm::mat4 aux = nodoCuerpo;
aux = glm::scale(aux, glm::vec3(4.2f, 2.3f, 2.1f));
DibujarCubo(uniformModel, uniformColor, aux, color);

glm::mat4 nodoCabeza = nodoCuerpo;
nodoCabeza = glm::translate(nodoCabeza, glm::vec3(2.55f, 0.55f, 0.0f));
```

Aquí se observa cómo se construyen partes más complejas a partir de una figura simple, modificando su tamaño y posición.

Después se agregan los detalles de la cabeza como hocico, ojos y nariz, utilizando transformaciones más pequeñas pero dependientes de la cabeza.

```
aux = nodoHocico;
aux = glm::scale(aux, glm::vec3(1.5f, 0.75f, 0.95f));
DibujarCubo(uniformModel, uniformColor, aux, color);

aux = nodoCabeza;
aux = glm::translate(aux, glm::vec3(0.55f, 0.12f, 0.82f));
DibujarEsfera(uniformModel, uniformColor, aux, color);
```

Esto permite añadir realismo al modelo sin perder la estructura jerárquica.

En la sección de la cola se implementa un sistema articulado con dos segmentos, permitiendo que tenga movimiento flexible.

```
glm::mat4 nodoCola1 = nodoCuerpo;
nodoCola1 = glm::rotate(nodoCola1, glm::radians(colaAng), glm::vec3(0.0f, 0.0f, 1.0f));

glm::mat4 nodoCola2 = nodoCola1;
nodoCola2 = glm::rotate(nodoCola2, glm::radians(colaAng2), glm::vec3(0.0f, 0.0f, 1.0f));
```

Esto es importante porque muestra cómo se pueden encadenar articulaciones para lograr movimientos más naturales.

Finalmente, las patas se construyen mediante una función que encapsula toda su estructura, aplicando rotaciones para simular articulaciones superiores e inferiores.

```
DibujarPata(nodoCuerpo, glm::vec3(1.35f, -0.90f, 0.78f), pataSuperior[0],
pataInferior[0], uniformModel, uniformColor);
```

Esto permite reutilizar la misma lógica para las cuatro patas, manteniendo consistencia en el modelo.

Las articulaciones del perro se controlan mediante el teclado numérico y algunas teclas adicionales:

- **Pata delantera izquierda**
  Superior: 7 / 8
  Inferior: 4 / 5

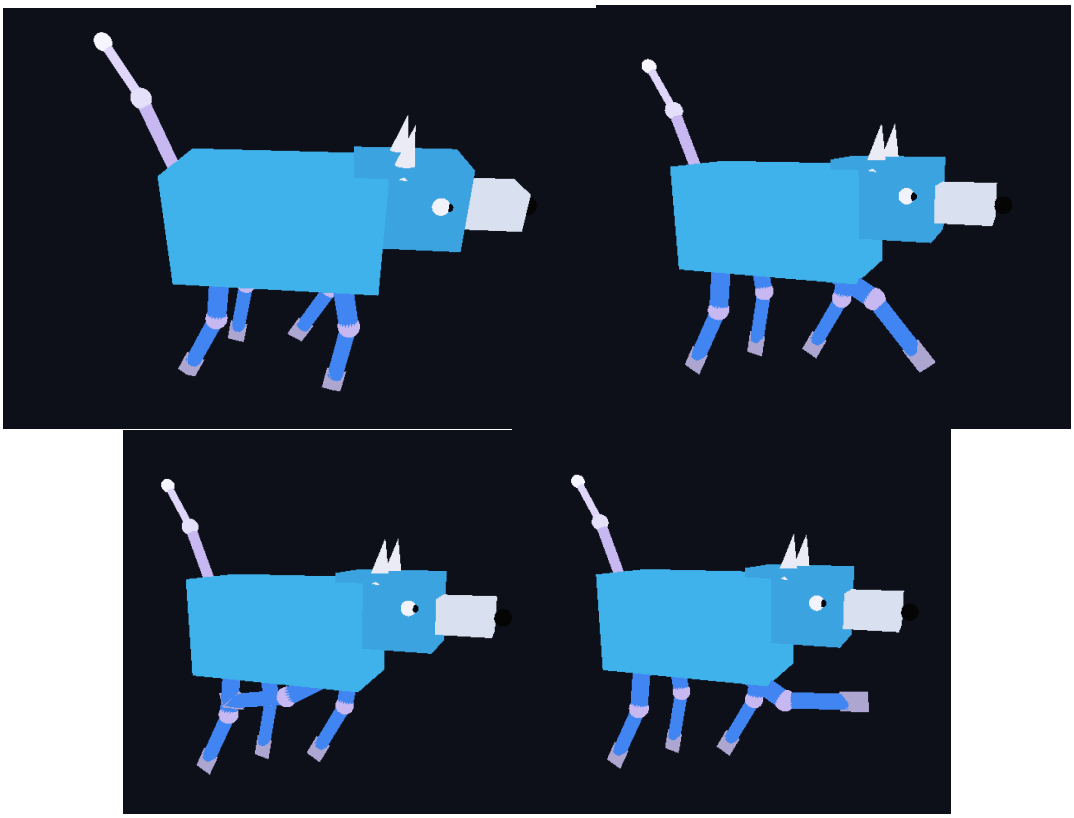
- **Pata delantera derecha**
  Superior: 9 / 6
  Inferior: 3 / .

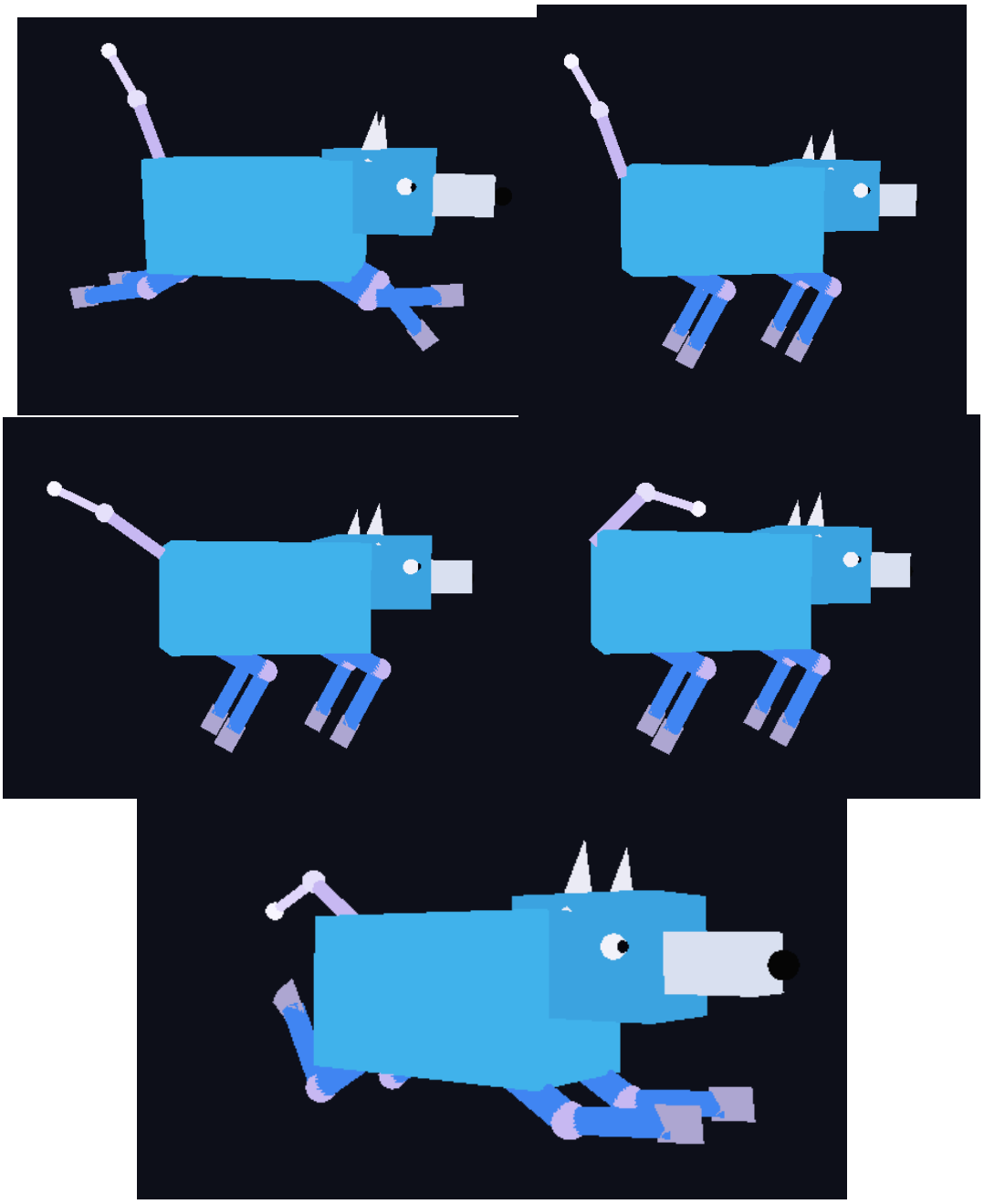
- **Pata trasera izquierda**
  Superior: + / -
  Inferior: * / /

- **Pata trasera derecha**
  Superior: Enter / 0
  Inferior: 1 / 2

- **Cola**
  Primera articulación: F / V
  Segunda articulación: R / C

Cada par de teclas permite mover la articulación en sentidos opuestos, logrando un control completo del movimiento del modelo.





Como resultado final, se obtiene un modelo 3D de un perro completamente articulado capaz de adoptar distintas posturas y simular movimiento, como caminar o correr.

2.- Liste los problemas que tuvo a la hora de hacer estos ejercicios y si los resolvió explicar cómo fue, en caso de error adjuntar captura de pantalla

Durante la realización de estos ejercicios se presentaron algunos problemas principalmente relacionados con el manejo de transformaciones y articulaciones. Uno de los principales inconvenientes fue que algunas partes del modelo, como las patas o la cola, no giraban en el punto correcto, lo que ocasionaba movimientos poco naturales. Este problema se resolvió ajustando el orden de las transformaciones (traslación antes de rotación) para asegurar que cada articulación funcionara respecto a su punto adecuado.

Otro problema fue que algunas piezas aparecían desalineadas o con escalas incorrectas, lo que afectaba la proporción del modelo. Esto se solucionó revisando los valores de escala y posición hasta lograr una estructura más coherente visualmente.

También se presentó dificultad al asignar controles de teclado, ya que algunas teclas no respondían como se esperaba. Esto se corrigió verificando correctamente la lectura de teclas y asegurando que los valores se actualizaran en cada frame.

En general, todos los problemas pudieron resolverse mediante pruebas, ajustes en los parámetros de transformación y revisión de la lógica del programa, logrando finalmente un modelo funcional y estable.

3.- Conclusión:

a. Los ejercicios del reporte: Complejidad, Explicación

Los ejercicios presentaron una complejidad media, ya que implicaron el uso de transformaciones jerárquicas y el manejo de múltiples articulaciones dentro de un mismo modelo. Aunque los conceptos base son claros, su implementación requiere atención al orden de las transformaciones y a la relación entre los distintos nodos del modelo. En general, los ejercicios permitieron comprender mejor cómo construir estructuras articuladas en 3D a partir de primitivas básicas.

b. Comentarios generales: Faltó explicar a detalle, ir más lento en alguna explicación, otros comentarios y sugerencias para mejorar desarrollo de la práctica

En algunos momentos hizo falta explicar con mayor detalle ciertos conceptos, especialmente el manejo de jerarquías y el orden de las transformaciones, ya que estos influyen directamente en el comportamiento del modelo. También sería útil que algunas explicaciones se realizaran de forma más pausada o con ejemplos adicionales, para facilitar la comprensión. Como sugerencia, incluir diagramas o representaciones visuales de las jerarquías ayudaría a entender mejor la relación entre las partes del modelo. Mas como diagramas de ejemplo en el classroom.

c. Conclusión

En conclusión, la práctica permitió reforzar los conocimientos sobre modelado geométrico y transformaciones en OpenGL, así como comprender la importancia de las estructuras jerárquicas en la creación de modelos articulados. Además, se desarrolló la capacidad de resolver problemas relacionados con el movimiento y la interacción, logrando un resultado funcional y visualmente coherente.

Bibliografía

El Baúl del Programador. (2016, January 1). *OpenGL - Prácticas Informática Gráfica: Modelos jerárquicos - Un Robot*. OpenGL - Prácticas Informática Gráfica: Modelos Jerárquicos - Un Robot. https://elbauldelprogramador.com/opengl-practicas-informatica-grafica-modelos-jerarquicos-un-robot/

Cervantes, H. (n.d.). *Tutorial OpenGL*. http://liim.izt.uam.mx/~humberto/itzamna/TUTORIAL/tutorial.html

Zavala, F. (2025, December 12). *Graphics pipeline en OpenGL*. Código En Llamas | Programación, Algoritmos Y Software Explicado Paso a Paso. https://codigoenllamas.com/graphics-pipeline-en-opengl